Designing a Highly Available Key-Value Store

Abstract

We describe the design of a distributed key-value store that favours availability over strong consistency. Keys are spread across nodes with consistent hashing, every write is replicated to N nodes, and reads and writes succeed once R and W replicas respond. Conflicts are detected with vector clocks and resolved by the client.

1. Partitioning

Each node owns several virtual positions on a hash ring. A key is stored on the first N distinct nodes found walking clockwise from its hash. Virtual nodes smooth out load when machines join or leave, and let more powerful machines take a larger share of the ring.

2. Replication

The node that receives a write acts as coordinator. It stores the value locally and forwards it to the next N-1 healthy nodes on the ring. With R + W > N, every read overlaps at least one replica that saw the latest write.

3. Handling failures

If a replica is unreachable, the write goes to the next healthy node with a hint naming the intended owner. When the owner recovers, the hinted data is handed back. Merkle trees let replicas find and repair differences cheaply in the background.

4. Versioning and conflicts

Because writes may be accepted by different replicas during a partition, a key can end up with several concurrent versions. Each version carries a vector clock; versions whose clocks are ordered can be discarded automatically, while truly concurrent ones are returned to the client to merge.

5. Membership

Nodes learn about each other through a gossip protocol. Every second each node exchanges its view of the ring with a random peer, so changes spread in O(log n) rounds without any central coordinator.

6. Evaluation

In production-like tests the store kept 99.9th percentile read and write latencies under 300 ms while serving peak traffic, and remained writable during simulated data-centre failures.

7. Lessons learned

Tunable N, R and W let each application choose its own balance of durability, latency and consistency. Pushing conflict resolution to clients keeps the storage layer simple, but requires careful application design.